Solution: Find all Duplicate Numbers

Problem Statement

We are given an unsorted array containing `n` numbers taken from the range `1 to n`. The array has some numbers appearing twice, **find all these duplicate numbers using constant space.**

Example 1:

```
Input: [3, 4, 4, 5, 5]
Output: [4, 5]
```

Example 2:

```
Input: [5, 4, 7, 2, 3, 5, 3]
Output: [3, 5]
```

Solution

This problem follows the **Cyclic Sort** pattern and shares similarities with `Find the Duplicate Number`. Following a similar approach, we will place each number at its correct index. After that, we will iterate through the array to find all numbers that are not at the correct indices. All these numbers are duplicates.

Code

Here is what our algorithm will look like:

Java

```java
import java.util.*;

class Solution {

  // Function to find duplicate numbers in an array.
  public static List<Integer> findNumbers(int[] nums) {
    int i = 0;
    while (i < nums.length) {
      if (nums[i] != nums[nums[i] - 1])  // Check if the current element is not in its correct position.
        swap(nums, i, nums[i] - 1);  // Swap the current element with the element at its correct position
      else
```

```
12            i++;  // Move to the next element if the current element is already in its correct position.
13        }
14
15        List<Integer> duplicateNumbers = new ArrayList<>();
16        for (i = 0; i < nums.length; i++) {
17          if (nums[i] != i + 1)  // Identify elements that are not in their correct positions, which are dupl
18            duplicateNumbers.add(nums[i]);  // Add the duplicates to the list.
19        }
20
21        return duplicateNumbers;
22      }
23
24      // Function to swap two elements in an array...
```

Run Code Result | Stdout

You must run your code first

Execute

Time Complexity

The time complexity of the above algorithm is O(n).

Space Complexity

Ignoring the space required for storing the duplicates, the algorithm runs in constant space O(1).

← Previous

Find All Duplicate Numbers (Easy)

Next →

Problem Challenge 1: Find The Corrupt Pair (Easy)

Mark as Completed